

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_5

5. Menus, Toolbars, and More

Joshua M. Willman¹

(1) Hampton, VA, USA

As you add more and more features to your applications, you will need some way to present all of the individual options to the user. A **menu** is a list of commands that a computer program can perform presented in a more manageable and organized fashion. No matter what type of device or application you are using or what kind of menu system it has, if it has a menu in place, its role is to help you navigate through the various operations in order to help you select the tasks you wish to perform.

Graphical user interfaces have numerous kinds of menus, such as context menus or pull-down menus, and can contain a variety of text and symbols. These symbols can be selected to give the computer an instruction. Think about a text editing program and all the various icons at the top, for example, open a file, save a file, select font, or select color. All of these symbols and text represent some task that the application can perform presented to the user in a manner that should promote better ease of use.

In this chapter we are going to take a look at how to create menus and toolbars for GUI applications in PyQt5. Everything up until now has been used to build a foundation for creating user interfaces, from PyQt classes and widgets to layout design.

Different from previous chapters, here we will be looking at how to make completely functioning programs, a notepad GUI and a simple photo editor GUI. These applications can either be used right away or as a starting point for building your own GUI program. There is a fair amount of information, from new concepts to additional widgets and classes, covered in this chapter.

For menus using PyQt, you will take a look at

1. The `QMainWindow` class for setting up the main window
2. Creating `QMenuBar` and `QMenu` objects
3. Adding actions to menus using the `QAction` class
4. Setting up the status bar using `QStatusBar` to display information about actions
5. Using `QDockWidget` to build detachable widgets to hold an application's tools
6. How to create submenus and checkable menu items

Other concepts and widgets covered include

- Setting and changing icons in the main window and on widgets with `QIcon`
- New types of dialogs including `QInputDialog`, `QColorDialog`, `QFontDialog`, and `QMessageBox`'s About dialog box
- How to handle and manipulate images using `QPixmap` and `QTransform` classes
- How to print images using `QPrinter` and `QPainter`

Let's jump right into coding a basic menu framework that will help you learn about creating menus with PyQt5 and some new classes, `QMainWindow` and `QMenu`.

Create a Basic Menu

For this first part, we will be taking a look at how to create a simple menubar. A **menubar** is a set of pull-down menus with list commands that we can use to interact with the program. In this program, the menubar will contain one menu with only one command, `Exit` (Listing 5-1).

```
# menu_framework.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QAction)
class BasicMenu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 350, 350) # x, y, width, height
        self.setWindowTitle('Basic Menu Example')
        self.createMenu()
        self.show()
    def createMenu(self):
        """
        Create skeleton application with a menubar
        """
        # Create actions for file menu
        exit_act = QAction('Exit', self)
        exit_act.setShortcut('Ctrl+Q')
        exit_act.triggered.connect(self.close)
        # Create menubar
        menu_bar = self.menuBar()
        menu_bar.setNativeMenuBar(False)
        # Create file menu and add actions
        file_menu = menu_bar.addMenu('File')
        file_menu.addAction(exit_act)

# Run program

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = BasicMenu()
    sys.exit(app.exec_())
```

Listing 5-1 Basic structure for creating the menu in an application

Figure 5-1 shows what adding a simple menu will look like on MacOS. Notice how in the left image `File` is displayed in the menubar, and when we scroll over it using our mouse, the `Exit` option is shown in the image on the right.

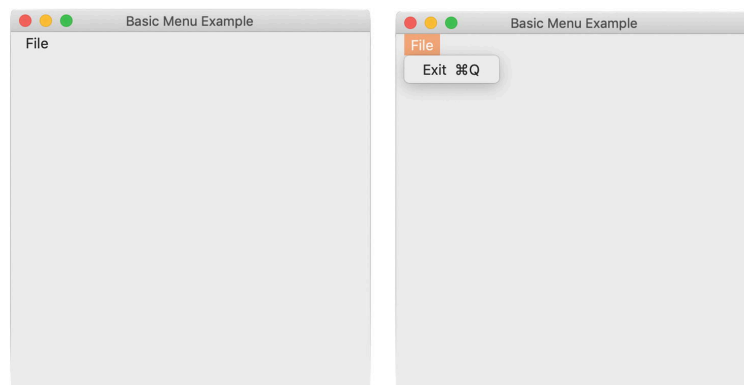


Figure 5-1 A menubar is created (left) displaying our `File` menu. A pull-down menu is displayed (right) with one command, `Exit`

Explanation

The framework for this program contains no widgets, but does show how to set up a simple `File` menu located in the top-left corner of the GUI. Take a look at the beginning of the program and notice the classes being imported from `QtWidgets`. We still import `QApplication`, but there are also two new classes, `QMainWindow` and `QAction`. You may also notice that this time there is no `QWidget`.

The `QMainWindow` class provides the necessary tools for building an application's graphical user interface. Notice that the `BasicMenu` class in the preceding code is written as

```
class BasicMenu(QMainWindow)
```

The class to build our window inherits from `QMainWindow` instead of `QWidget`.

`QMainWindow` vs. `QWidget`

The `QMainWindow` class focuses on creating and managing the layout for the main window of an application. It allows you to set up a window with a status bar, a toolbar, dock widgets, or other menu options in pre-defined places all designed around functions that the main window should have.

The `QWidget` class is the base class for all user interface objects in Qt. The widget is the basic building block of GUIs. It is interactive, allowing the user to communicate with the computer to perform some task. Many of the widgets you have already looked at, such as `QPushButton` and `QTextEdit`, are just subclasses of `QWidget` that give functionality to your programs.

A window in an application is really just a widget that is not embedded in a parent widget. What is important to understand is that `QMainWindow` actually inherits from the `QWidget` class. It is a special purpose class focusing mainly on

creating menus and housing widgets in your program. In Figure 5-2, you can see how the different widgets that QMainWindow can use have areas specifically assigned for them. Take a look at the image to see how the menubar, dock widgets, and the central widget can be arranged inside of the main window.

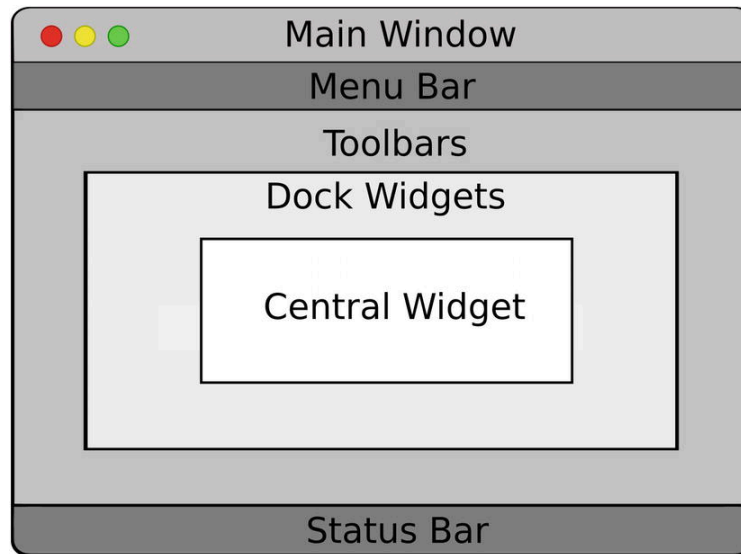


Figure 5-2 Example layout for QMainWindow class (Adapted from <https://doc.qt.io/> web site)

The central widget in the center of the window must be set if you are going to use QMainWindow as your base class. For example, you could use a single QTextEdit widget or create a QWidget object to act as a parent to a number of other widgets, then use `setCentralWidget()`, and set your central widget for the main window.

Creating the Menubar and Adding Actions

At the top of the window in Figure 5-1, you will see the menubar which contains one menu, `File`. In order to create a menubar, you must create an instance of the `QMenuBar` class, which we created by

```
menu_bar = self.menuBar()
```

You could create a menubar by actually calling the `QMenuBar` class, but it is just as easy to create a menubar using the `menuBar()` function provided by QMainWindow.

Note Due to guidelines set by the MacOS system, you must set the property to use the platform's native settings to `False`. Otherwise, the menu will not appear in the window. You can do this with `menu_bar.setNativeMenuBar(False)`. For those using Windows or Linux, you can comment this line out or delete it completely from your code.

Adding menus to the menubar is also really simple in PyQt:

```
file_menu = menu_bar.addMenu('File')
```

Here we are using the `addMenu()` method to add a menu named `File` to the `menu_bar`. Using `addMenu()` adds a `QMenu` object to our menubar.

Once again, it is just as simple to use the functions provided by the `QMainWindow` class.

A menu contains a list of action items such as Open, Close, and Find. In PyQt, these actions are created from the **QAction** class, defining actions for menus and toolbars. Many actions in an application are also given shortcut keys making it easier to perform that task, for example, Ctrl+V for the paste action (Cmd+V on MacOS) or Ctrl+X for the cut action (Cmd+X on MacOS). Take a look at how the Exit action is created and then added to `file_menu`.

```
exit_act = QAction('Exit', self)
exit_act.setShortcut('Ctrl+Q')
exit_act.triggered.connect(self.close)
```

The Exit action, `exit_act`, is an instance of the `QAction` class. In the next line the shortcut for the `exit_act` is set explicitly using the `setShortcut()` method with the key combination Ctrl+Q. Another way to set the shortcut is to use the ampersand key, `&`, in front of the letter you want to use as the shortcut. For example,

```
open_act = QAction('&Open', self)
```

Note By default, on MacOS shortcuts are disabled. The best way to use them is with `setShortcut()`.

Similar to `QPushButtons`, actions in the menu emit a signal and need to be connected to a slot in order to perform an action. This is done using `triggered.connect()`. Using the `QAction` class is very useful since many common commands can be invoked through the menu, toolbars, or shortcuts and need to be able to perform correctly no matter which widget invokes the action.

Setting Icons with the QIcon Class

In GUI applications, icons are small graphical images or symbols that can represent an action the user can perform. They are often used to help the user more quickly locate common actions and better navigate an application. For example, in a word editing program such as Microsoft Word, the toolbar at the top of the GUI contains a large amount of icons, each with icon and textual descriptions.

Chapter 2 briefly introduced the `QPixmap` class which is used for handling images. The **QIcon** class provides methods that can use pixmaps and modify their style or size to be used in an application. One really great use of `QIcon` is to set the appearance of an icon representing an action to active or disabled.

Setting icons is very useful not only for the actions in a toolbar but also for setting the application icon that is displayed in the title bar of the GUI window. Actions can be in four states, represented by icons: Normal, Disabled, Active, or Selected. `QIcon` can also be used when setting the icons on other widgets, as well.

Listing 5-2 shows how to reset the application icon displayed in the main window and how to set the icon on a QPushButton.

NoteFor MacOS users, the application window cannot be changed due to system guidelines. You should still take a look at this program though, as it also shows how to set icons for other widgets in PyQt.

```
# change_icons.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QLabel, QWidget, QPushButton, QVBoxLayout)
from PyQt5.QtGui import QIcon
from PyQt5.QtCore import QSize
import random

class ChangeIcon(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()

    def initializeUI(self):
        self.setGeometry(100, 100, 200, 200)
        self.setWindowTitle('Set Icons Example')
        self.setWindowIcon(QIcon('images/pyqt_logo.png'))
        self.createWidgets()
        self.show()

    def createWidgets(self):
        """
        Set up widgets.
        """
        info_label = QLabel("Click on the button and select a fruit.")
        self.images = [
            "images/1_apple.png",
            "images/2_pineapple.png",
            "images/3_watermelon.png",
            "images/4_banana.png"
        ]
        self.icon_button = QPushButton(self)
        self.icon_button.setIcon(QIcon(random.choice(self.images)))
        self.icon_button.setIconSize(QSize(60, 60))
        self.icon_button.clicked.connect(self.changeButtonIcon)
        # Create vertical layout and add widgets
        v_box = QVBoxLayout()
        v_box.addWidget(info_label)
        v_box.addWidget(self.icon_button)
        # Set main layout of window
        self.setLayout(v_box)

    def changeButtonIcon(self):
        """
        When the button is clicked, change the icon to one of the images in the list.
        """
        self.icon_button.setIcon(QIcon(random.choice(self.images)))
        self.icon_button.setIconSize(QSize(60, 60))

# Run program

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = ChangeIcon()
    sys.exit(app.exec_())
```

Listing 5-2 Code to show how to set icons for the main window and on QPushButton

You can see what the application should look like on Windows in Figure 5-3. The application icon normally displayed in the top-left corner is changed to the PyQt logo in the right image. Notice how the application icon is missing in the MacOS version in Figure 5-4.

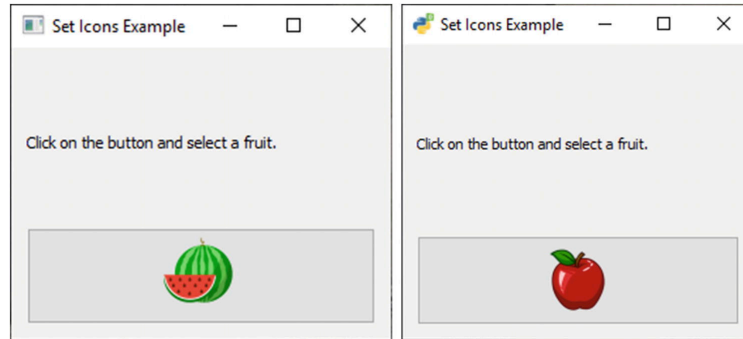


Figure 5-3 The original application icon in the top-left corner of the window (left) can be set to new a new icon (right) using the `setWindowIcon()` method. On Windows and Linux systems, changing the icon isn't an issue

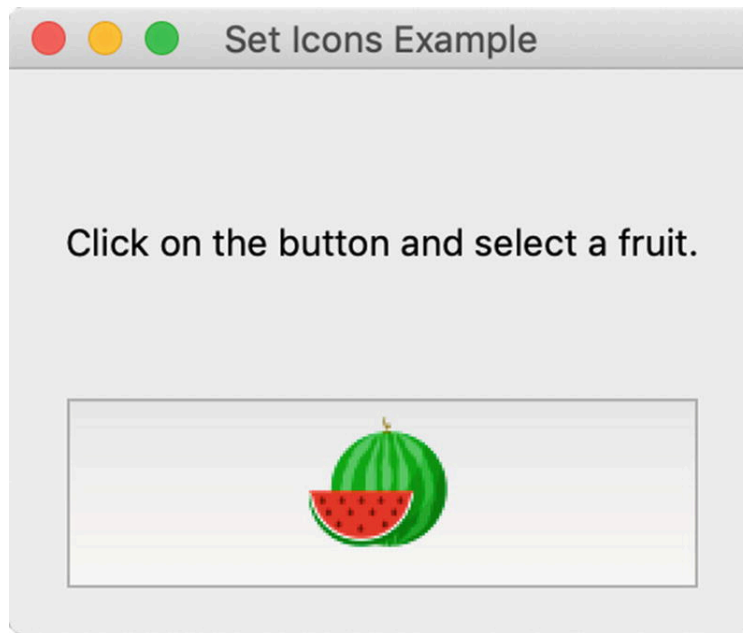


Figure 5-4 The application icon is not displayed in the title area on MacOS systems

Explanation

The preceding example contains a simple button that, when clicked, will select an image randomly from the `images` list.

Setting the main window's application icon can be done by calling the `setWindowIcon()` method and setting as the argument for `QIcon` the location of the image file. This can be seen in the following line:

```
self.setWindowIcon(QIcon('images/pyqt_logo.png'))
```

If a widget is created that can display an icon, then calling the `setIcon()` method on that widget will allow you to display an image on it.

```
icon_button.setIcon(QIcon(random.choice(self.images)))  
icon_button.setIconSize(QSize(60, 60))
```

Here, the icon for `icon_button` is chosen randomly and passed as an argument to be handled by `QIcon`. Calling the `setIconSize()` method on a widget can be used to change the size of the icon. PyQt will handle the sizing and style of the widget based on your parameters in the main window. The button is then connected to a slot that is used to change the icon.

Finally, the label and button widgets are arranged using `QVBoxLayout` and set as the main window's layout.

Project 5.1 – Rich Text Notepad GUI

For the first project, let's take a look at how to improve the notepad GUI we saw back in Chapter 4. It's important to actually build a complete program to help you to learn how to make your own GUIs from start to finish.

This time we will add a proper menubar with menus and actions. The user will also have the ability to open and save their text, either as HTML or plain text, and edit the text's font, color, or size to give more functionality and creativity to their notes.

Figure 5-5 shows an example of the completed application with text of different sizes, colors, fonts, and highlights.

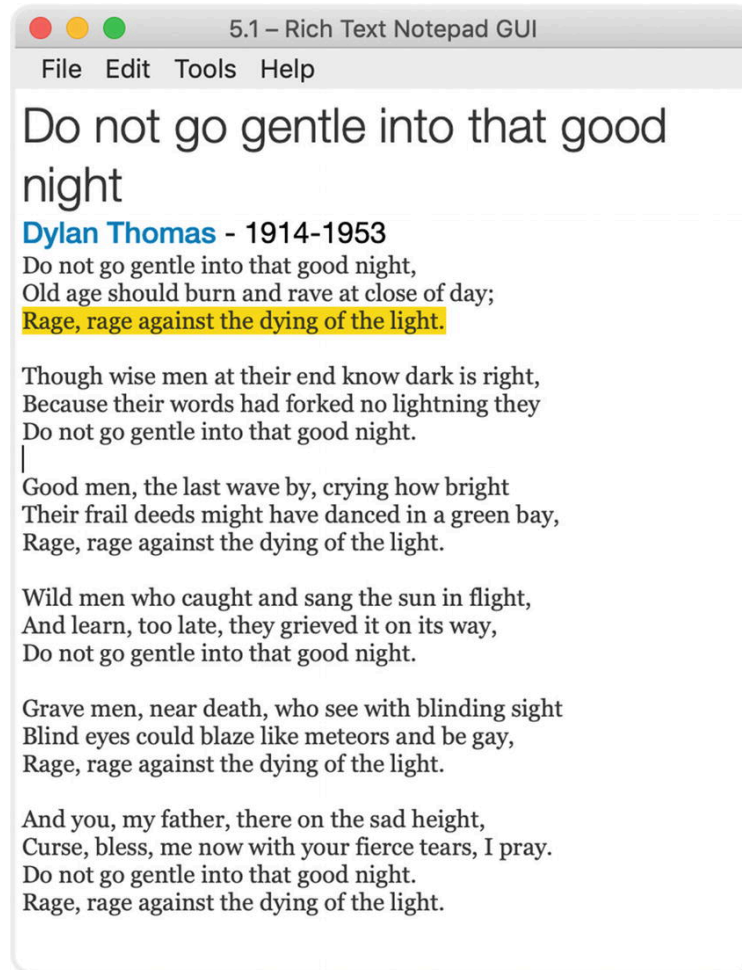


Figure 5-5 Notepad GUI with menubar and QTextEdit widget

Design the Rich Text Notepad GUI

Usually before creating interfaces, you should think about and map out what kind of functionality you want your application to have and what kind of widgets you might need in order to achieve those tasks.

For a text editing application, the layout is generally very basic – a menubar at the top of the window with different menus for the various functions and tools, and an area for displaying and editing text. For the text field, we will be using a QTextEdit widget which will also serve as the central widget for the QMainWindow object.

This application will consist of four menus in the menubar – File, Edit, Tools, and Help. Having different menus in the menubar can help to organize actions under different categories as well as help the user to more easily locate actions they want to use. Take a look at Figure 5-6 to see the various menu items that will be included in this project.

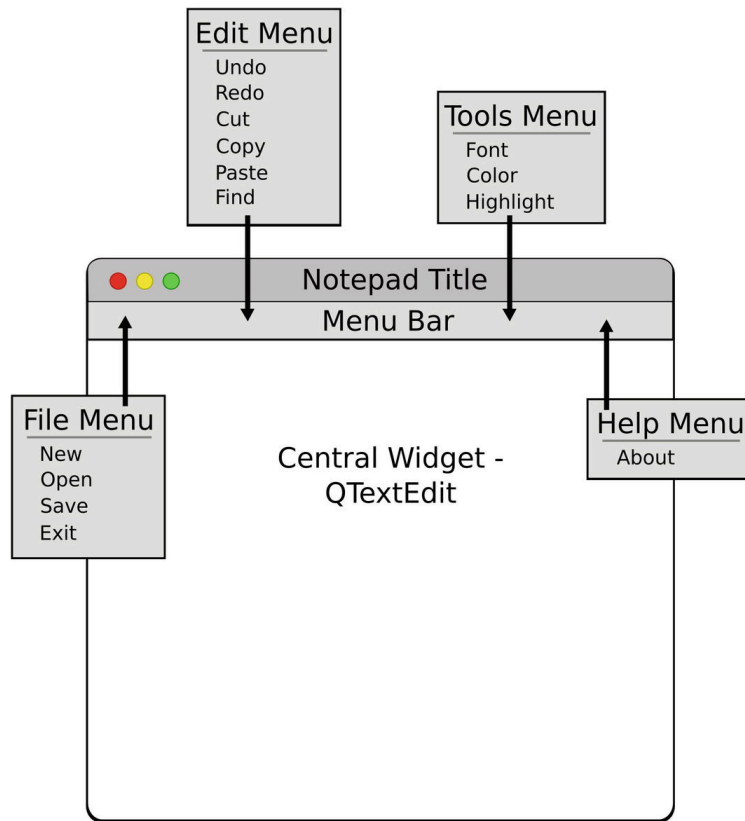


Figure 5-6 Design showing the layout for the notepad GUI and the different menus and actions

More Types of Dialog Boxes in PyQt

In this project there are a number of different dialog boxes native to PyQt that are used including `QInputDialog`, `QFileDialog`, `QFontDialog`, `QColorDialog`, and `QMessageBox`. Let's take a moment to get familiar with some new types of dialog boxes and find out how to include them in our code.

The `QInputDialog` Class

`QInputDialog` is a native dialog box in PyQt that can be used to receive input from the user. The input is a single value that can be a string, a number, or an item from a list.

To create an input dialog and get text from the user:

```
find_text, ok = QInputDialog.getText(self, "Search Text", "Find:")
```

In this example, shown in Figure 5-7, an input dialog object is created by calling `QInputDialog`. The `getText()` method takes a single string input from the user. The second argument, "Search Text", is the title for the dialog and `Find:` is the message displayed in the dialog box. An input dialog returns two values – the input from the user and a Boolean value. If the OK button is clicked, then the `ok` variable is set to `True`.

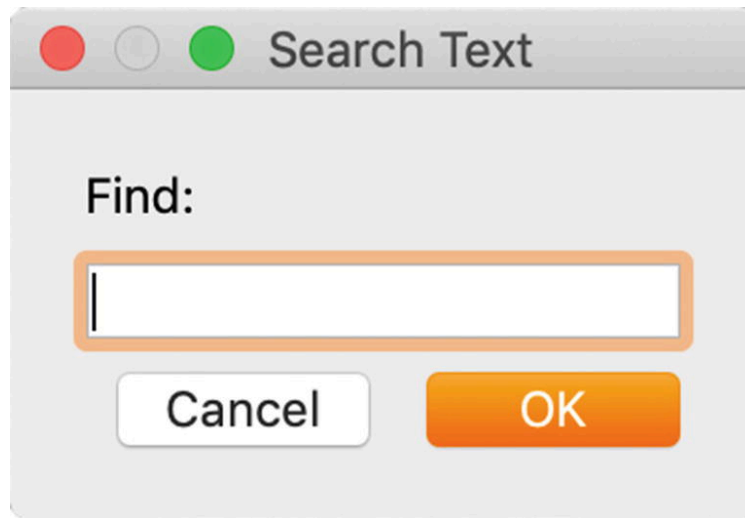


Figure 5-7 Example of `QInputDialog` dialog box

For other types of input, you can use one of the following methods:

- `getMultiLineText()` – Method to get a multiline string from the user
- `getInt()` – Method to get an integer from the user
- `getDouble()` – Method to get a floating-point number from the user
- `getItem()` – Method to let the user select an item from a list of strings

The `QFontDialog` Class

`QFontDialog` provides a dialog box that allows the user to select and manipulate different types of fonts. To create a font dialog box and choose a font, use the `getFont()` method. The font dialog that is native to PyQt is shown in Figure [5-8](#).

```
font, ok = QFontDialog.getFont()
```

The `font` keyword is the particular font returned from `getFont()` and `ok` is a Boolean variable to check whether the user selected a font and clicked the OK button.

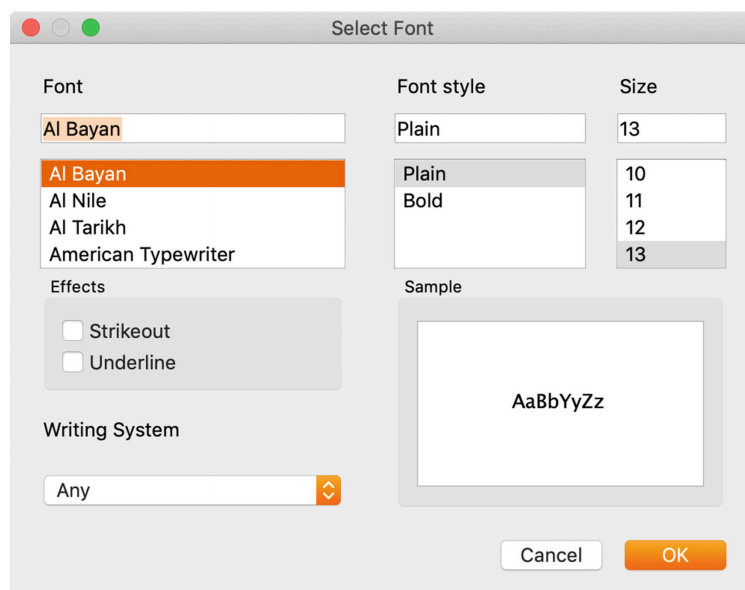


Figure 5-8 QFontDialog dialog box

When the user clicks OK, a font is selected. However, if Cancel is clicked, then the initial font is returned. If you have a default font that you would like to use in case the user does not select OK, you could do the following:

```
font, ok = QFontDialog.getFont(QFont("Helvetica", 10), self)
self.text_edit_widget.setCurrentFont(font)
```

In order to change the font if a new one has been chosen, use the `setCurrentFont()` method and change it to the new font.

The QColorDialog Class

The **QColorDialog** class creates a dialog box for selecting colors like the one in Figure 5-9. Selecting colors can be useful for changing the color of the text, a window's background color, and many other actions.

To create a color dialog box and select a color, use the following line of code:

```
color = QColorDialog.getColor()
```

Then check if the user selected a color and clicked the OK button by using the `isValid()` method. If so, you could use `setTextColor()` to change the color of the text or `setBackgroundColor()` to change the color of the background.

```
if color.isValid():
    self.text_field.setTextColor(color)
```

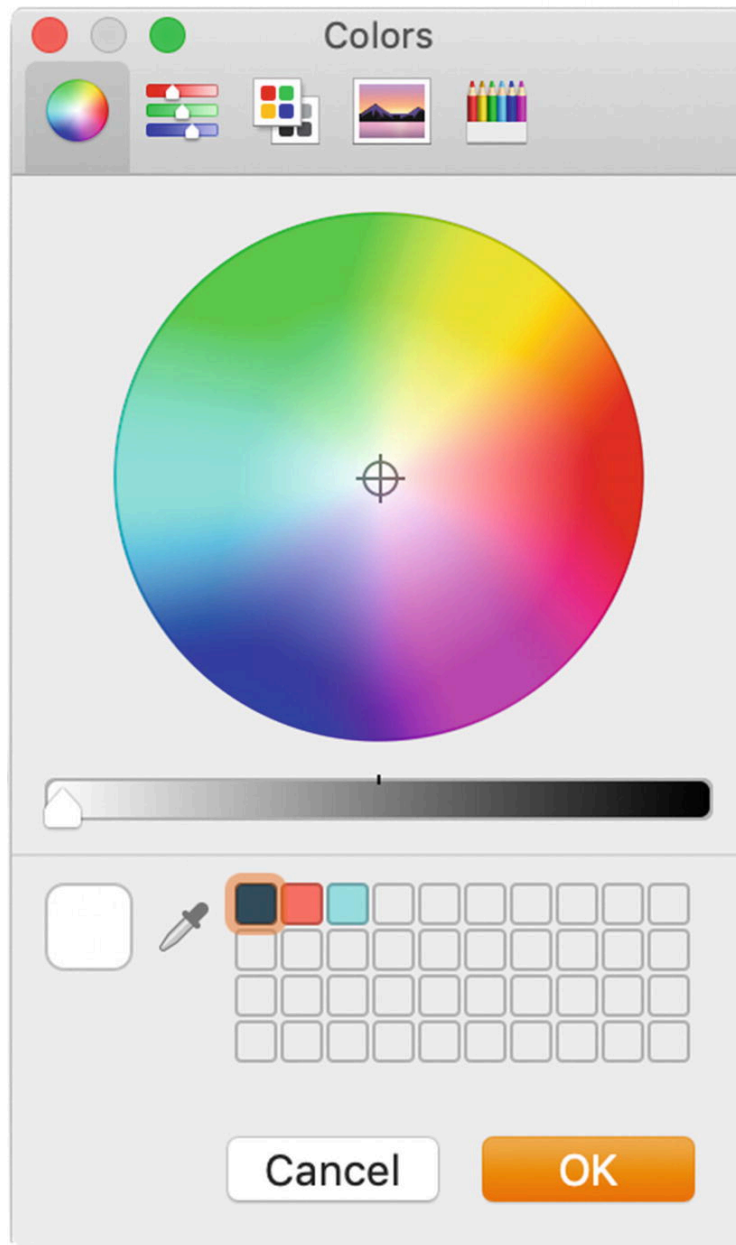


Figure 5-9 QColorDialog dialog box

The About Dialog Box

In many applications you can often find an `About` item in the menu. Clicking this item will open a dialog box that displays information about the application such as the software's logo, title, latest version number, and other legal information.

The `QMessageBox` class that we looked at in Chapter 3 also provides an `about()` method for creating a dialog for displaying a title and text. To create an about dialog box like the one in Figure 5-10, try

```
QMessageBox.about(self, "About Notepad", "Beginner's Practical Guide to PyQt\n\nProject 5.1 - Notepad G
```

You can also display an application icon in the window. If an icon is not provided, the `about()` method will try and find one from parent widget. To provide an icon, call the `setWindowIcon()` method on the `QApplication` object in the program's `main()` method.

```
app.setWindowIcon(QIcon("images/app_logo.png"))
```

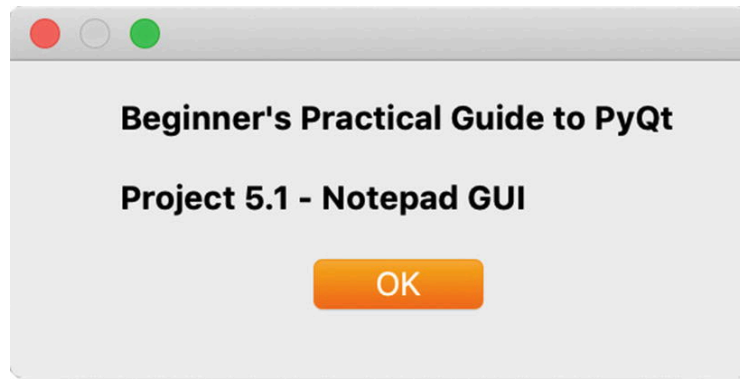


Figure 5-10 Example About dialog box from the notepad GUI

Rich Text Notepad GUI Solution

The `QTextEdit` widget already provides functionality for writing in either plain text or rich text formats. In this program, you will explore how to use the different methods of `QTextEdit`, such as `undo()` and `redo()`, as well as the different dialog classes to create a notepad application. This program also allows you to save your text in either plain text format or HTML format if you want to preserve the rich text (Listing 5-3).

```
# richtext_notepad.py
#Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QAction, QMessageBox, QTextEdit, QFileDialog, Q
from PyQt5.QtGui import QIcon, QTextCursor, QColor
from PyQt5.QtCore import Qt
class Notepad(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 400, 500)
        self.setWindowTitle('5.1 - Rich Text Notepad GUI')
        self.createNotepadWidget()
        self.notepadMenu()
        self.show()
    def createNotepadWidget(self):
        """
        Set the central widget for QMainWindow, which is the QTextEdit widget for the notepad
        """
        self.text_field = QTextEdit()
        self.setCentralWidget(self.text_field)
    def notepadMenu(self):
        """
        Create menu for notepad GUI
        """
```

```

# Create actions for file menu
new_act = QAction(QIcon('images/new_file.png'), 'New', self)
new_act.setShortcut('Ctrl+N')
new_act.triggered.connect(self.clearText)
open_act = QAction(QIcon('images/open_file.png'), 'Open', self)
open_act.setShortcut('Ctrl+O')
open_act.triggered.connect(self.openFile)
save_act = QAction(QIcon('images/save_file.png'), 'Save', self)
save_act.setShortcut('Ctrl+S')
save_act.triggered.connect(self.saveToFile)
exit_act = QAction(QIcon('images/exit.png'), 'Exit', self)
exit_act.setShortcut('Ctrl+Q')
exit_act.triggered.connect(self.close)
# Create actions for edit menu
undo_act = QAction(QIcon('images/undo.png'), 'Undo', self)
undo_act.setShortcut('Ctrl+Z')
undo_act.triggered.connect(self.text_field.undo)
redo_act = QAction(QIcon('images/redo.png'), 'Redo', self)
redo_act.setShortcut('Ctrl+Shift+Z')
redo_act.triggered.connect(self.text_field.redo)
cut_act = QAction(QIcon('images/cut.png'), 'Cut', self)
cut_act.setShortcut('Ctrl+X')
cut_act.triggered.connect(self.text_field.cut)
copy_act = QAction(QIcon('images/copy.png'), 'Copy', self)
copy_act.setShortcut('Ctrl+C')
copy_act.triggered.connect(self.text_field.copy)
paste_act = QAction(QIcon('images/paste.png'), 'Paste', self)
paste_act.setShortcut('Ctrl+V')
paste_act.triggered.connect(self.text_field.paste)
find_act = QAction(QIcon('images/find.png'), 'Find', self)
find_act.setShortcut('Ctrl+F')
find_act.triggered.connect(self.findTextDialog)
# Create actions for tools menu

font_act = QAction(QIcon('images/font.png'), 'Font', self)
font_act.setShortcut('Ctrl+T')
font_act.triggered.connect(self.chooseFont)
color_act = QAction(QIcon('images/color.png'), 'Color', self)
color_act.setShortcut('Ctrl+Shift+C')
color_act.triggered.connect(self.chooseFontColor)
highlight_act = QAction(QIcon('images/highlight.png'), 'Highlight', self)
highlight_act.setShortcut('Ctrl+Shift+H')
highlight_act.triggered.connect(self.chooseFontBackgroundColor)
about_act = QAction('About', self)
about_act.triggered.connect(self.aboutDialog)
# Create menubar
menu_bar = self.menuBar()
menu_bar.setNativeMenuBar(False)
# Create file menu and add actions
file_menu = menu_bar.addMenu('File')
file_menu.addAction(new_act)
file_menu.addSeparator()
file_menu.addAction(open_act)
file_menu.addAction(save_act)
file_menu.addSeparator()
file_menu.addAction(exit_act)
# Create edit menu and add actions
edit_menu = menu_bar.addMenu('Edit')
edit_menu.addAction(undo_act)

```

```

edit_menu.addAction(redo_act)
edit_menu.addSeparator()
edit_menu.addAction(cut_act)
edit_menu.addAction(copy_act)
edit_menu.addAction(paste_act)
edit_menu.addSeparator()
edit_menu.addAction(find_act)
# Create tools menu and add actions

tool_menu = menu_bar.addMenu('Tools')
tool_menu.addAction(font_act)
tool_menu.addAction(color_act)
tool_menu.addAction(highlight_act)
# Create help menu and add actions
help_menu = menu_bar.addMenu('Help')
help_menu.addAction(about_act)

def openFile(self):
    """
    Open a text or html file and display its contents in
    the text edit field.
    """
    file_name, _ = QFileDialog.getOpenFileName(self, "Open File",
        "", "HTML Files (*.html);;Text Files (*.txt)")
    if file_name:
        with open(file_name, 'r') as f:
            notepad_text = f.read()
            self.text_field.setText(notepad_text)
    else:
        QMessageBox.information(self, "Error",
            "Unable to open file.", QMessageBox.Ok)

def saveToFile(self):
    """
    If the save button is clicked, display dialog asking user if
    they want to save the text in the text edit field to a text file.
    """
    file_name, _ = QFileDialog.getSaveFileName(self, 'Save File',
        "", "HTML Files (*.html);;Text Files (*.txt)")
    if file_name.endswith('.txt'):
        notepad_text = self.text_field.toPlainText()
        with open(file_name, 'w') as f:
            f.write(notepad_text)
    elif file_name.endswith('.html'):
        notepad_richtext = self.text_field.toHtml()
        with open(file_name, 'w') as f:
            f.write(notepad_richtext)
    else:
        QMessageBox.information(self, "Error",
            "Unable to save file.", QMessageBox.Ok)

def clearText(self):
    """
    If the new button is clicked, display dialog asking user if
    they want to clear the text edit field or not.
    """
    answer = QMessageBox.question(self, "Clear Text",
        "Do you want to clear the text?", QMessageBox.No | QMessageBox.Yes,
        QMessageBox.Yes)
    if answer == QMessageBox.Yes:
        self.text_field.clear()
    else:

```



```

pass

def findTextDialog(self):
    """
    Search for text in QTextEdit widget
    """
    # Display input dialog to ask user for text to search for
    find_text, ok = QInputDialog.getText(self, "Search Text", "Find:")
    extra_selections = []
    # Check to make sure the text can be modified
    if ok and not self.text_field.isReadOnly():
        # set the cursor in the textedit field to the beginning
        self.text_field.moveCursor(QTextCursor.Start)
        color = QColor(Qt.yellow)
        # Look for next occurrence of text
        while(self.text_field.find(find_text)):
            # Use ExtraSelections to mark the text you are
            # searching for as yellow
            selection = QTextEdit.ExtraSelection()
            selection.format.setBackground(color)
            # Set the cursor of the selection
            selection.cursor = self.text_field.textCursor()
            # Add selection to list
            extra_selections.append(selection)
        # Highlight selections in text edit widget
        for i in extra_selections:
            self.text_field.setExtraSelections(extra_selections)

def chooseFont(self):
    """
    Select font for text
    """
    current = self.text_field.currentFont()
    font, ok = QFontDialog.getFont(current, self, options=QFontDialog.DontUseNativeDialog)
    if ok:
        self.text_field.setCurrentFont(font) # Use setFont() to set all text to one type of font

def chooseFontColor(self):
    """
    Select color for text
    """
    color = QColorDialog.getColor()
    if color.isValid():
        self.text_field.setTextColor(color)

def chooseFontBackgroundColor(self):
    """
    Select color for text's background
    """
    color = QColorDialog.getColor()
    if color.isValid():
        self.text_field.setTextBackgroundColor(color)

def aboutDialog(self):
    """
    Display information about program dialog box
    """
    QMessageBox.about(self, "About Notepad", "Beginner's Practical Guide to PyQt\n\nProject 5.1 - N
# Run program

```

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = Notepad()
    sys.exit(app.exec_())
```

Listing 5-3 Rich text notepad GUI code

Your program with its menubar and different menus should look similar to the images in Figure 5-11.

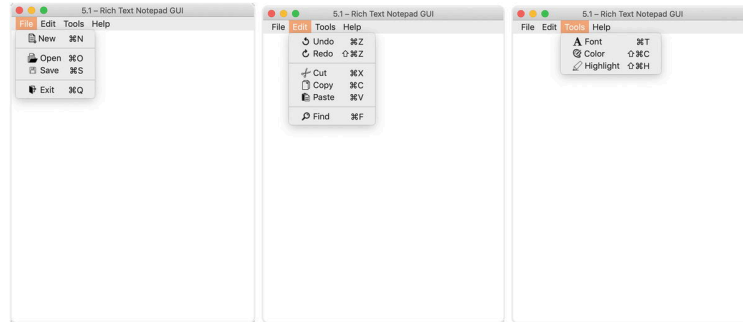


Figure 5-11 The notepad GUI with its different menus displayed, File menu (left), Edit menu (middle), and Tools menu (right)

Explanation

There are quite a few classes to import for the notepad application. From the `QtWidgets` module, we need to import `QMainWindow` and `QAction` for creating the menubar and menu items. We also need to include the different PyQt dialog classes such as `QFileDialog` and `QInputDialog`. From `QtGui`, which provides many of the basic classes for creating GUI applications, `QIcon` is used for handling icons, `QTextCursor` can be used to get information about the cursor in text documents, and `QColor` provides methods to create colors in PyQt.

The main window is initialized in the `Notepad` class which inherits from `QMainWindow`. The `createNotepadWidget()` method creates a `QTextEdit` widget and sets it as the central widget for the `QMainWindow` using

```
self.setCentralWidget(self.text_field)
```

Next, the `notepadMenu()` method sets up the menubar object along with the different menu items. The menu for the notepad application, which can be seen in Figure 5-10, contains four menus, `File`, `Edit`, `Tools`, and `Help`. Each menu is given its own menu items for the most part based on guidelines that we have come to expect from applications. For example, a general `File` menu creates actions that allow the user to open, save, import, export, or print files.

The following bit of code shows how to create the action to open a file:

```
open_act = QAction(QIcon('images/open_file.png'), 'Open', self)
open_act.setShortcut('Ctrl+O')
open_act.triggered.connect(self.openFile)
```

The `open_act` object is generated by the `QAction` class. `QIcon` is used to set an icon next to the action's text in the menu. Then the action is given text to display, `Open`. Many of the actions in the notepad program are given a textual shortcut using `setShortcut()`. Finally, we connect the `open_act` signal that is produced when it is triggered to a slot, in this case the `openFile()` method. Other actions are created in a similar manner.

`QTextEdit` already has predefined slots, such as `cut()`, `copy()`, and `paste()`, to interact with text. For most of the actions in the `Edit` menu, their signals are connected to these special slots rather than creating new ones.

Once all the actions are defined, the `menu_bar` is created and the different menus are created by using the `addMenu()` method.

```
file_menu = menu_bar.addMenu('File')
```

Each of the actions is added to a menu by calling the `addAction()` method on the appropriate menu. To add a divider between categories in a menu, use `addSeparator()`.

```
file_menu.addAction(new_act)
file_menu.addSeparator()
```

There are a number of functions that are called on when a menu item is clicked. Each one of them opens a dialog box and returns some kind of input from the user, such as a new file, text or background color, or a keyword from a text search using the `QInputDialog` class.

Project 5.2 – Simple Photo Editor GUI

With the introduction of smartphones that have the latest technology to take amazing photos, more pictures are taken and modified every day. However, not every picture is perfect as soon as it is taken and technology also gives us tools to edit those images to our liking. Some photo editors are very simple, allowing the user to rotate, crop, or add text to images. Others let the user change the contrast and exposure, reduce noise, or even add special effects.

In the following project, we will be taking a look at how to create a basic image editor, Figure [5-12](#), that can give you a foundation to build your own application.

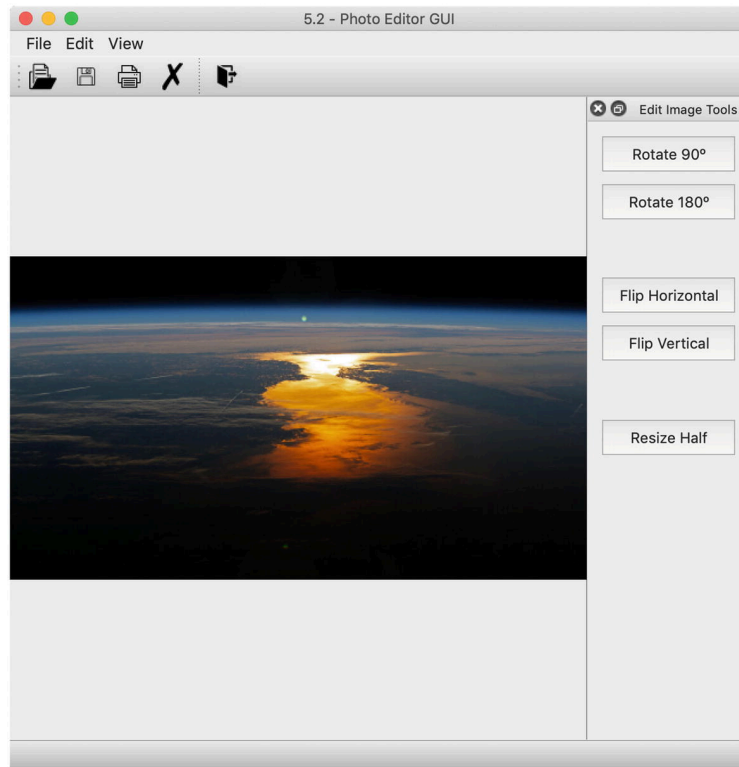


Figure 5-12 Photo editor GUI displaying the menubar at the top, the toolbar with icons underneath the menubar, the central widget which displays the image, the status bar at the bottom, and the dock widget on the right containing simple tools for editing the photo. Earth photo from [nasa.org](https://www.nasa.gov)

Design the Photo Editor GUI

Similar to Project 5.1, this GUI will also have a menubar that will contain various menus – `File`, `Edit`, and `View`. The layout for this project can be seen in Figure 5-13. Under the menubar is the toolbar created using the `QToolBar` class which contains icons that represent actions the user can take such as open a file, save a file, and print. This project will also introduce the `QDockWidget` class for creating widgets that can be docked inside the main window or left floating, the `QStatusBar` class for displaying information to the user, and checkable menu items which we will use to hide or show the dock widget.

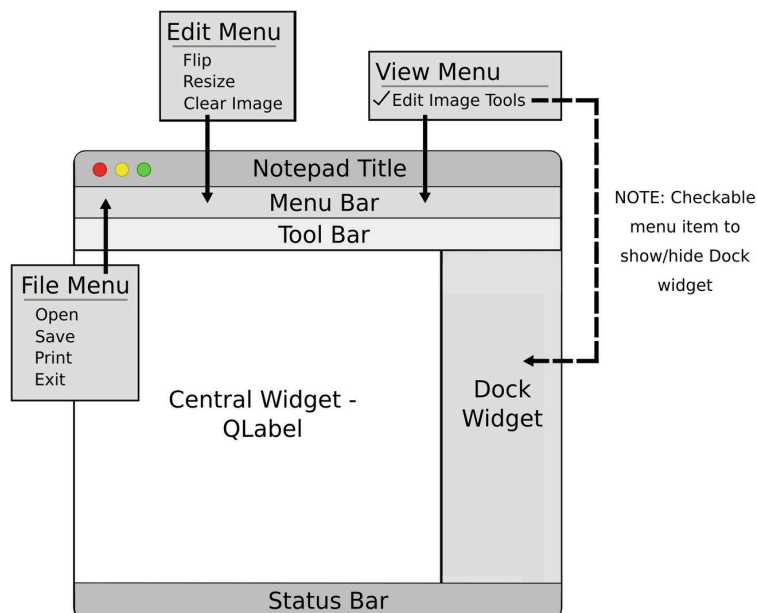


Figure 5-13 Layout for the photo editor GUI. The main window is much busier than before containing a toolbar, a dock widget, and a status bar

QDockWidget, QStatusBar, and More

Let's take a look at some of the important features that will be introduced in the photo editor program:

- The `QDockWidget` class
- The `QStatusBar` class
- Creating submenus
- Creating checkable menu items

Listing 5-4 creates a more detailed GUI framework that demonstrates these concepts.

```
# menu_framework2.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QStatusBar, QAction, QTextEdit, QToolBar, QDock
from PyQt5.QtCore import Qt, QSize
from PyQt5.QtGui import QIcon
class BasicMenu(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen
        """
        self.setGeometry(100, 100, 350, 350) # x, y, width, height
        self.setWindowTitle('Basic Menu Example 2')
        # Set central widget for main window
        self.setCentralWidget(QTextEdit())
        self.createMenu()
        self.createToolBar()
        self.createDockWidget()
        self.show()
    def createMenu(self):
        """
        Create menubar and menu actions
        """
        # Create actions for file menu
        self.exit_act = QAction(QIcon('images/exit.png'), 'Exit', self)
        self.exit_act.setShortcut('Ctrl+Q')
        self.exit_act.setStatusTip('Quit program')
        self.exit_act.triggered.connect(self.close)
        # Create actions for view menu
        full_screen_act = QAction('Full Screen', self, checkable=True)
        full_screen_act.setStatusTip('Switch to full screen mode')
        full_screen_act.triggered.connect(self.switchToFullScreen)
        # Create menubar
        menu_bar = self.menuBar()
        menu_bar.setNativeMenuBar(False)
        # Create file menu and add actions
        file_menu = menu_bar.addMenu('File')
        file_menu.addAction(self.exit_act)
        # Create view menu, Appearance submenu, and add actions
        view_menu = menu_bar.addMenu('View')
        appearance_submenu = view_menu.addMenu('Appearance')
        appearance_submenu.addAction(full_screen_act)
        # Display info about tools, menu, and view in the status bar
```

```

        self.setStatusBar(QStatusBar(self))
    def createToolBar(self):
        """
        Create toolbar for GUI

        """
        # Set up toolbar
        tool_bar = QToolBar("Main Toolbar")
        tool_bar.setIconSize(QSize(16, 16))
        self.addToolBar(tool_bar)
        # Add actions to toolbar
        tool_bar.addAction(self.exit_act)
    def createDockWidget(self):
        """
        Create dock widget

        """
        # Set up dock widget
        dock_widget = QDockWidget()
        dock_widget.setWindowTitle("Example Dock")
        dock_widget.setAllowedAreas(Qt.AllDockWidgetAreas)
        # Set main widget for the dock widget
        dock_widget.setWidget(QTextEdit())
        # Set initial location of dock widget in main window

        self.addDockWidget(Qt.LeftDockWidgetArea, dock_widget)
    def switchToFullScreen(self, state):
        """
        If state is True, then display the main window in full screen.
        Otherwise, return the window to normal.

        """
        if state:
            self.showFullScreen()
        else:
            self.showNormal()
# Run program
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = BasicMenu()
    sys.exit(app.exec_())

```

Listing 5-4 Code to demonstrate how to create dock widgets, status bars, and toolbars

Your GUI created from this program should look similar to the one in Figure

5-14.

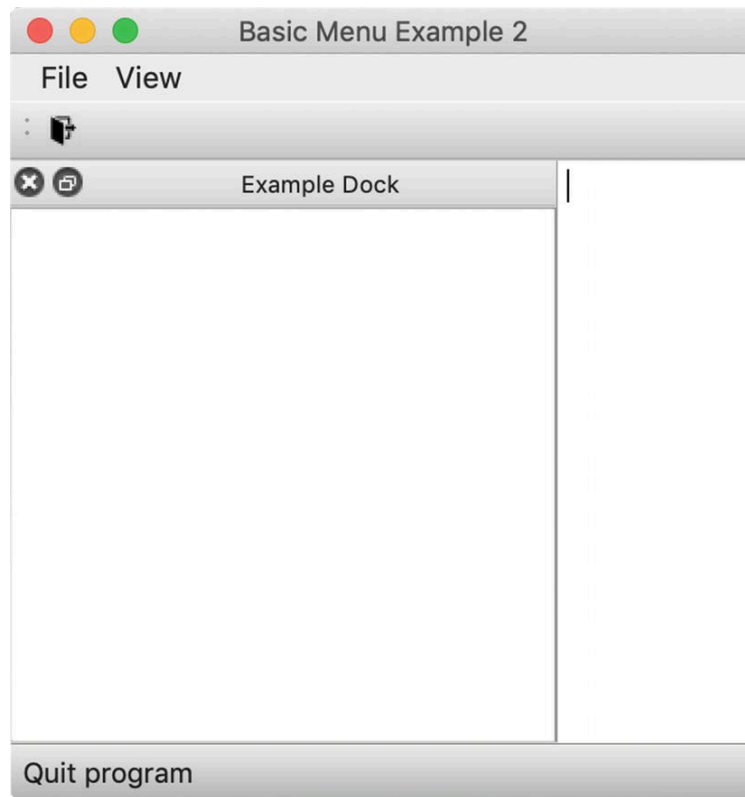


Figure 5-14 Framework program for creating GUIs with toolbars, status bars, and dock widgets. The status bar on the bottom displays the text “Quit program” when the mouse hovers over the Exit icon in the toolbar

Explanation

A few new classes from the `QtWidgets` module are imported including `QStatusBar`, `QToolBar`, and `QDockWidget`. Getting to learn a little bit about these classes will be useful for creating more complex GUIs.

The `QStatusBar` Class

At the bottom of the GUI in Figure 5-14, there is a horizontal bar with the text “Quit program” displayed inside of it. This bar is known as the status bar as is created from the `QStatusBar` class. Sometimes an icon’s or menu item’s function is not explicitly understood. This widget is very useful for displaying extra information to the user about the capabilities of an action.

To create a status bar object, you can use the `setStatusBar()` method which is part of the `QMainWindow` class. To create an empty status bar, pass `QStatusBar` as an argument.

```
self.setStatusBar(QStatusBar(self))
```

The first time this method is called, it creates the status bar, and following calls will return the status bar object.

In order to display a message in the status bar when the mouse hovers over an icon, you need to call the `setStatusTip()` method on an action object. For example:

```
exit_act.setStatusTip('Quit program')
```

will display the text “Quit program” when the mouse is over the `exit_act` icon or menu command.

To display text in the status bar when the program begins or when a function is called, use the `showMessage()` method.

```
self.statusBar().showMessage('Welcome back!')
```

The QToolBar Class

When the user is performing a number of routine tasks, having to open up the menu to select an action multiple times can become tedious. Luckily, the **QToolBar** class provides ways to create a toolbar with icons, text, or standard Qt widgets for quick access to frequently used commands.

Toolbars are generally located under the menubar like in Figure 5-14, but can also be placed vertically or at the bottom of the main window above the status bar. Refer to the image in Figure 5-2 for an idea of arranging the different widgets in the main window.

A GUI can only have one menubar but it can have multiple toolbars. To create a toolbar object, create an instance of the `QToolBar` class and give it a title and then add it to the main window using `MainWindow`’s `addToolBar()` method.

```
tool_bar = QToolBar("Main Toolbar")
tool_bar.setIconSize(QSize(16, 16))
self.addToolBar(tool_bar)
```

You should set the size of the icons in the toolbar using the `setIconSize()` method with `QSize()` to avoid extra padding when PyQt tries to figure out the arrangement by itself.

To add an action to the toolbar, use `addAction()`:

```
tool_bar.addAction(self.exit_act)
```

If you need to add widgets in your toolbar, you should also use `QAction` to take advantage of the classes’ ability to handle multiple interface elements.

The QDockWidget Class

The **QDockWidget** class is used to create detachable or floating tool palettes or widget panels. Dock widgets are secondary windows that provide additional functionality to GUI windows.

To create the dock widget object, create an instance of `QDockWidget` and set the widget’s title using the `setWindowTitle()` method.

```
dock_widget = QDockWidget()
```



```
dock_widget.setWindowTitle("Example Dock")
```

When the dock widget is docked inside of the main window, PyQt handles the resizing of the dock window and the central widget. You can also specify the areas you want the dock to be placed in the main window using `setAllowedAreas()`.

```
dock_widget.setAllowedAreas(Qt.AllDockWidgetAreas)
```

In the preceding line of code, the dock widget can be placed on any of the four sides of the window. To limit the allowable dock areas, use the following Qt methods:

- `LeftDockWidgetArea`
- `RightDockWidgetArea`
- `TopDockWidgetArea`
- `BottomDockWidgetArea`

The dock widget can act as a parent for a single widget using `setWidget()`.

```
dock_widget.setWidget(QTextEdit())
```

In order to place multiple widgets inside the dock, you could use a single `QWidget` as the parent for multiple child widgets and arrange them using one of the layout managers from Chapter 4. Then, pass that `QWidget` as the argument to `setWidget()`.

Finally, to set the initial location of the dock widget in the main window, use

```
self.addDockWidget(Qt.LeftDockWidgetArea, dock_widget)
```

In this application if the dock widget is closed, we cannot get it back. In the “Photo Editor GUI Solution,” we will take a look at how to use checkable menu items to hide or show the dock widget.

Creating Submenus with Checkable Menu Items

When an application becomes very complex and filled with actions, its menus can also begin to turn into a cluttered mess. Using **submenus**, we can organize similar categories together and simplify the menu system. Figure 5-15 displays an example of a submenu.

Similar to creating a regular menu, use the `addMenu()` method to create submenus.

```
view_menu = menu_bar.addMenu('View')
appearance_submenu = view_menu.addMenu('Appearance')
appearance_submenu.addAction(full_screen_act)
```

Here we first create the `View` menu and add it to the menubar. The `appearance_submenu` is then created and added to the `View` menu. Don’t forget to also add an action to the submenu using the `addAction()` method.

The `appearance_submenu` in the example has a `full_screen_act` action added to it that allows the user to switch between full screen and normal screen modes. Menu items can also be created so that they act just like switches, being able to be turned on and off. To set an action as checkable, include the option `checkable=True` in the `QAction` parameters.

```
full_screen_act = QAction('Full Screen', self, checkable=True)
```

Then, when the action is clicked, it will send a signal and you can use a slot to check the state of the menu item, whether it is on or off. This could be useful for showing or hiding dock widgets or the status bar.

To make the action checked and active from the start, you can call the `trigger()` method on the action.

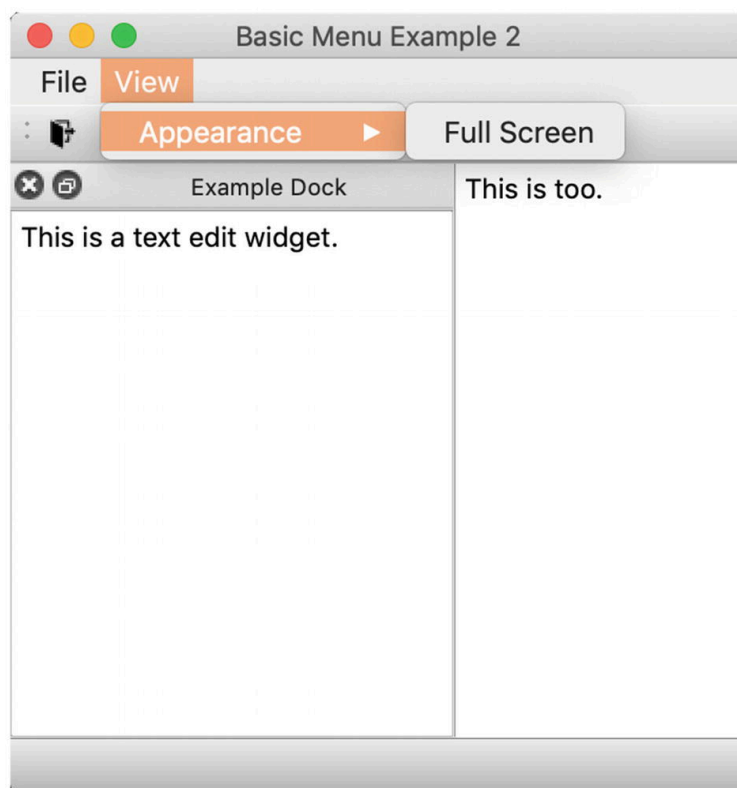


Figure 5-15 Example submenu that also contains a checkable action to switch between full screen and normal modes

Photo Editor GUI Solution

Now that we have gone over how to set up the different types of menus, we can finally get started on coding the photo editor application (Listing 5-5).

```
# photo_editor.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QLabel, QAction, QFileDialog, QDesktop
from PyQt5.QtGui import QIcon, QPixmap, QTransform, QPainter
from PyQt5.QtCore import Qt, QSize, QRect
from PyQt5.QtPrintSupport import QPrinter, QPrintDialog
class PhotoEditor(QMainWindow):
    def __init__(self):
```

```

super().__init__()
self.initializeUI()
def initializeUI(self):
    """
    Initialize the window and display its contents to the screen
    """
    self.setFixedSize(650, 650)
    self.setWindowTitle('5.2 - Photo Editor GUI')
    self.centerMainWindow()
    self.createToolsDockWidget()
    self.createMenu()
    self.createToolBar()
    self.photoEditorWidgets()
    self.show()
def createMenu(self):
    """
    Create menu for photo editor GUI
    """
    # Create actions for file menu
    self.open_act = QAction(QIcon('images/open_file.png'), "Open", self)
    self.open_act.setShortcut('Ctrl+O')
    self.open_act.setStatusTip('Open a new image')
    self.open_act.triggered.connect(self.openImage)
    self.save_act = QAction(QIcon('images/save_file.png'), "Save", self)
    self.save_act.setShortcut('Ctrl+S')
    self.save_act.setStatusTip('Save image')
    self.save_act.triggered.connect(self.saveImage)
    self.print_act = QAction(QIcon('images/print.png'), "Print", self)
    self.print_act.setShortcut('Ctrl+P')
    self.print_act.setStatusTip('Print image')
    self.print_act.triggered.connect(self.printImage)
    self.print_act.setEnabled(False)
    self.exit_act = QAction(QIcon('images/exit.png'), 'Exit', self)
    self.exit_act.setShortcut('Ctrl+Q')
    self.exit_act.setStatusTip('Quit program')
    self.exit_act.triggered.connect(self.close)
    # Create actions for edit menu

    self.rotate90_act = QAction("Rotate 90°", self)
    self.rotate90_act.setStatusTip('Rotate image 90° clockwise')
    self.rotate90_act.triggered.connect(self.rotateImage90)
    self.rotate180_act = QAction("Rotate 180°", self)
    self.rotate180_act.setStatusTip('Rotate image 180° clockwise')
    self.rotate180_act.triggered.connect(self.rotateImage180)
    self.flip_hor_act = QAction("Flip Horizontal", self)
    self.flip_hor_act.setStatusTip('Flip image across horizontal axis')
    self.flip_hor_act.triggered.connect(self.flipImageHorizontal)
    self.flip_ver_act = QAction("Flip Vertical", self)
    self.flip_ver_act.setStatusTip('Flip image across vertical axis')
    self.flip_ver_act.triggered.connect(self.flipImageVertical)
    self.resize_act = QAction("Resize Half", self)
    self.resize_act.setStatusTip('Resize image to half the original size')
    self.resize_act.triggered.connect(self.resizeImageHalf)
    self.clear_act = QAction(QIcon('images/clear.png'), "Clear Image", self)
    self.clear_act.setShortcut('Ctrl+D')
    self.clear_act.setStatusTip('Clear the current image')
    self.clear_act.triggered.connect(self.clearImage)
    # Create menubar
    menu_bar = self.menuBar()

```

```

menu_bar.setNativeMenuBar(False)
# Create file menu and add actions
file_menu = menu_bar.addMenu('File')
file_menu.addAction(self.open_act)
file_menu.addAction(self.save_act)
file_menu.addSeparator()
file_menu.addAction(self.print_act)
file_menu.addSeparator()
file_menu.addAction(self.exit_act)
# Create edit menu and add actions
edit_menu = menu_bar.addMenu('Edit')
edit_menu.addAction(self.rotate90_act)
edit_menu.addAction(self.rotate180_act)
edit_menu.addSeparator()
edit_menu.addAction(self.flip_hor_act)
edit_menu.addAction(self.flip_ver_act)
edit_menu.addSeparator()
edit_menu.addAction(self.resize_act)
edit_menu.addSeparator()
edit_menu.addAction(self.clear_act)
# Create view menu and add actions
view_menu = menu_bar.addMenu('View')
view_menu.addAction(self.toggle_dock_tools_act)
# Display info about tools, menu, and view in the status bar
self.setStatusBar(QStatusBar(self))
def createToolBar(self):
    """
    Create toolbar for photo editor GUI

    """
    tool_bar = QToolBar("Photo Editor Toolbar")
    tool_bar.setIconSize(QSize(24,24))
    self.addToolBar(tool_bar)
    # Add actions to toolbar
    tool_bar.addAction(self.open_act)
    tool_bar.addAction(self.save_act)
    tool_bar.addAction(self.print_act)
    tool_bar.addAction(self.clear_act)
    tool_bar.addSeparator()
    tool_bar.addAction(self.exit_act)
def createToolsDockWidget(self):
    """
    Use View -> Edit Image Tools menu and click the dock widget on or off.
    Tools dock can be placed on the left or right of the main window.
    """
    # Set up QDockWidget

    self.dock_tools_view = QDockWidget()
    self.dock_tools_view.setWindowTitle("Edit Image Tools")
    self.dock_tools_view.setAllowedAreas(Qt.LeftDockWidgetArea |
        Qt.RightDockWidgetArea)
    # Create container QWidget to hold all widgets inside dock widget
    self.tools_contents = QWidget()
    # Create tool push buttons
    self.rotate90 = QPushButton("Rotate 90°")
    self.rotate90.setMinimumSize(QSize(130, 40))
    self.rotate90.setStatusTip('Rotate image 90° clockwise')

```

```

self.rotate90.clicked.connect(self.rotateImage90)
self.rotate180 = QPushButton("Rotate 180°")
self.rotate180.setMinimumSize(QSize(130, 40))
self.rotate180.setStatusTip('Rotate image 180° clockwise')
self.rotate180.clicked.connect(self.rotateImage180)
self.flip_horizontal = QPushButton("Flip Horizontal")
self.flip_horizontal.setMinimumSize(QSize(130, 40))
self.flip_horizontal.setStatusTip('Flip image across horizontal axis')
self.flip_horizontal.clicked.connect(self.flipImageHorizontal)
self.flip_vertical = QPushButton("Flip Vertical")
self.flip_vertical.setMinimumSize(QSize(130, 40))
self.flip_vertical.setStatusTip('Flip image across vertical axis')
self.flip_vertical.clicked.connect(self.flipImageVertical)
self.resize_half = QPushButton("Resize Half")
self.resize_half.setMinimumSize(QSize(130, 40))
self.resize_half.setStatusTip('Resize image to half the original size')
self.resize_half.clicked.connect(self.resizeImageHalf)
# Set up vertical layout to contain all the push buttons
dock_v_box = QVBoxLayout()
dock_v_box.addWidget(self.rotate90)
dock_v_box.addWidget(self.rotate180)
dock_v_box.addStretch(1)
dock_v_box.addWidget(self.flip_horizontal)
dock_v_box.addWidget(self.flip_vertical)
dock_v_box.addStretch(1)
dock_v_box.addWidget(self.resize_half)
dock_v_box.addStretch(6)
# Set the main layout for the QWidget, tools_contents,
# then set the main widget of the dock widget

self.tools_contents.setLayout(dock_v_box)
self.dock_tools_view.setWidget(self.tools_contents)
# Set initial location of dock widget
self.addDockWidget(Qt.RightDockWidgetArea, self.dock_tools_view)
# Handles the visibility of the dock widget
self.toggle_dock_tools_act = self.dock_tools_view.toggleViewAction()
def photoEditorWidgets(self):
    """
    Set up instances of widgets for photo editor GUI
    """
    self.image = QPixmap()
    self.image_label = QLabel()
    self.image_label.setAlignment(Qt.AlignCenter)
    # Use setSizePolicy to specify how the widget can be resized, horizontally and vertically. Here
    self.image_label.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Ignored)
    self.setCentralWidget(self.image_label)
def openImage(self):
    """
    Open an image file and display its contents in label widget.
    Display error message if image can't be opened.
    """
    image_file, _ = QFileDialog.getOpenFileName(self, "Open Image", "",
        "JPG Files (*.jpeg *.jpg );;PNG Files (*.png);;Bitmap Files (*.bmp);;\n"
        "GIF Files (*.gif)")
    if image_file:
        self.image = QPixmap(image_file)
        self.image_label.setPixmap(self.image.scaled(self.image_label.size(),
            Qt.KeepAspectRatio, Qt.SmoothTransformation))
    else:

```

```

        QMessageBox.information(self, "Error",
                                "Unable to open image.", QMessageBox.Ok)
    self.print_act.setEnabled(True)
def saveImage(self):
    """
    Save the image.
    Display error message if image can't be saved.
    """
    image_file, _ = QFileDialog.getSaveFileName(self, "Save Image", "",
        "JPG Files (*.jpeg *.jpg );;PNG Files (*.png);;Bitmap Files (*.bmp);;\n"
        "GIF Files (*.gif)")
    if image_file and self.image.isNull() == False:
        self.image.save(image_file)
    else:
        QMessageBox.information(self, "Error",
                                "Unable to save image.", QMessageBox.Ok)
def printImage(self):
    """
    Print image.
    """
    # Create printer object and print output defined by the platform

    # the program is being run on.
    # QPainter.NativeFormat is the default
    printer = QPainter()
    printer.setOutputFormat(QPrinter.NativeFormat)
    # Create printer dialog to configure printer
    print_dialog = QPrintDialog(printer)
    # If the dialog is accepted by the user, begin printing
    if (print_dialog.exec_() == QPrintDialog.Accepted):
        # Use QPainter to output a PDF file
        painter = QPainter()
        # Begin painting device
        painter.begin(printer)
        # Set QRect to hold painter's current viewport, which
        # is the image_label
        rect = QRect(painter.viewport())
        # Get the size of image_label and use it to set the size
        # of the viewport
        size = QSize(self.image_label.pixmap().size())
        size.scale(rect.size(), Qt.KeepAspectRatio)
        painter.setViewport(rect.x(), rect.y(), size.width(), size.height())
        painter.setWindow(self.image_label.pixmap().rect())
        # Scale the image_label to fit the rect source (0, 0)
        painter.drawPixmap(0, 0, self.image_label.pixmap())
        # End painting
        painter.end()
def clearImage(self):
    """
    Clears current image in QLabel widget
    """
    self.image_label.clear()
    self.image = QPixmap() # reset pixmap so that isNull() = True
def rotateImage90(self):
    """
    Rotate image 90° clockwise
    """
    if self.image.isNull() == False:
        transform90 = QTransform().rotate(90)

```

```

        pixmap = QPixmap(self.image)
        rotated = pixmap.transformed(transform90, mode=Qt.SmoothTransformation)
        self.image_label.setPixmap(rotated.scaled(self.image_label.size(),
            Qt.KeepAspectRatio, Qt.SmoothTransformation))
        self.image = QPixmap(rotated)
        self.image_label.repaint() # repaint the child widget
    else:
        # No image to rotate
        pass
def rotateImage180(self):
    """
    Rotate image 180° clockwise
    """
    if self.image.isNull() == False:
        transform180 = QTransform().rotate(180)
        pixmap = QPixmap(self.image)
        rotated = pixmap.transformed(transform180, mode=Qt.SmoothTransformation)
        self.image_label.setPixmap(rotated.scaled(self.image_label.size(),
            Qt.KeepAspectRatio, Qt.SmoothTransformation))
        # In order to keep being allowed to rotate the image, set the rotated image as self.image
        self.image = QPixmap(rotated)
        self.image_label.repaint() # repaint the child widget
    else:
        # No image to rotate

        pass
def flipImageHorizontal(self):
    """
    Mirror the image across the horizontal axis
    """
    if self.image.isNull() == False:
        flip_h = QTransform().scale(-1, 1)
        pixmap = QPixmap(self.image)
        flipped = pixmap.transformed(flip_h)
        self.image_label.setPixmap(flipped.scaled(self.image_label.size(),
            Qt.KeepAspectRatio, Qt.SmoothTransformation))
        self.image = QPixmap(flipped)
        self.image_label.repaint()
    else:
        # No image to flip
        pass
def flipImageVertical(self):
    """
    Mirror the image across the vertical axis
    """
    if self.image.isNull() == False:
        flip_v = QTransform().scale(1, -1)
        pixmap = QPixmap(self.image)
        flipped = pixmap.transformed(flip_v)
        self.image_label.setPixmap(flipped.scaled(self.image_label.size(),
            Qt.KeepAspectRatio, Qt.SmoothTransformation))
        self.image = QPixmap(flipped)
        self.image_label.repaint()
    else:
        # No image to flip

        pass

```

```

def resizeImageHalf(self):
    """
    Resize the image to half its current size.
    """
    if self.image.isNull() == False:
        resize = QTransform().scale(0.5, 0.5)
        pixmap = QPixmap(self.image)
        resized = pixmap.transformed(resize)
        self.image_label.setPixmap(resized.scaled(self.image_label.size(),
            Qt.KeepAspectRatio, Qt.SmoothTransformation))
        self.image = QPixmap(resized)
        self.image_label.repaint()
    else:
        # No image to resize
        pass
def centerMainWindow(self):
    """
    Use QDesktopWidget class to access information about your screen
    and use it to center the application window.
    """
    desktop = QDesktopWidget().screenGeometry()
    screen_width = desktop.width()
    screen_height = desktop.height()
    self.move((screen_width - self.width()) / 2, (screen_height - self.height()) / 2)
# Run program
if __name__ == '__main__':
    app = QApplication(sys.argv)
    app.setAttribute(Qt.AA_DontShowIconsInMenus, True)
    window = PhotoEditor()
    sys.exit(app.exec_())

```

Listing 5-5 Photo editor code

Once complete, your application should look similar to the one in Figure [5-12](#).

Explanation

The photo editor application imports an assortment of new classes from different modules. From `QtWidgets` there are two new classes, `QDesktopWidget` and `QSizePolicy`. The **`QDesktopWidget`** class is used to access information about the screen on your computer. We will use it later to learn how to center a GUI on your desktop. The **`QSizePolicy`** class is used for resizing widgets.

From the `QtGui` module, we use **`QPixmap`** for handling images, **`QTransform`** for performing transformations on images, and **`QPainter`** which is useful for drawing, painting, and printing.

`QRect`, from `QtCore`, is used for creating rectangles. This will be used in the `printImage()` method.

The **`QPrintSupport`** module and its classes provide cross-platform support for accessing printers and printing documents.

The window is initialized like before except this time the `setFixedSize()` method is used to set the window's geometry so that it cannot be resized.

All of the menus, actions, icons, and status tips are also created in the `createMenu()` method. One important concept to note is that in this application only the toolbar displays icons, not in the menu. This is set with

```
app.setAttribute(Qt.AA_DontShowIconsInMenus, True)
```

The `File` menu contains the `Open`, `Save`, `Print`, and `Exit` actions. Setting the `setEnabled()` method on the `print_act` to `False` shows a disabled menu item and icon in the toolbar. The `print_act` only becomes enabled after an image is opened in the `openImage()` method.

Handling Images in PyQt

The `Edit` menu contains tools for rotating, flipping, resizing, and clearing images.

```
self.image = QPixmap(image_file)
self.image_label.setPixmap(self.image.scaled(self.image_label.size(), Qt.KeepAspectRatio, Qt.Sm
```

When an image file is opened using `QFileDialog`, we create a `QPixmap` object using that image, and then `setPixmap()` is called on the `image_label` to scale and set the image in the `QLabel` widget. Finally, the label is set as the central widget in the main window and resized according to the parameters in the `setSizePolicy()` method. `QPixmap` and other classes to handle images are covered further in Chapter 9.

The `QTransform` class provides a number of methods to use transformations on images. The photo editor application provides five actions for manipulating images: `Flip 90°`, `Flip 180°`, `Flip Horizontal`, `Flip Vertical`, and `Resize Half`. Figure 5-16 displays an example of an image being rotated 90°.

The tools located in the `Edit` menu could also be located in the toolbar. Instead, they are placed in a dock widget which contains push buttons with the different actions as an example of how to create a dock widget. The dock widget can also be toggled on and off in the `View` menu. To handle when the dock widget is checked or unchecked in the menu or if the user has closed the dock widget with the close button, use the `QDockWidget` method `toggleViewAction()`.

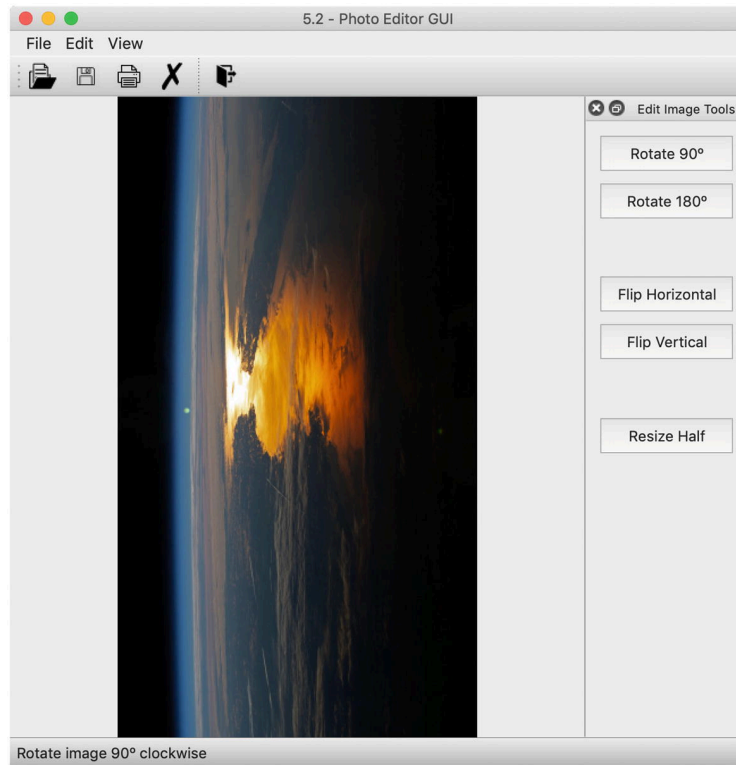


Figure 5-16 Example of 90° rotation in the photo editor GUI. The image is stretched horizontally to fit in the main window

The QPrinter Class

If you need to create a printing method for your applications, the photo editor includes a function adopted from the Qt document web site.¹

Take a look at the code and the comments to see how to use the QPrinter class to set up the printer and QPainter to set up the PDF file to be printed.

Center GUI Application on Your Desktop

The following bit of code shows how to use the **QDesktopWidget** class to find out information about the screen you are using in order to center the widget on the screen when the application starts:

```
desktop = QDesktopWidget().screenGeometry() # Create QDesktopWidget
screen_width = desktop.width() # Get screen width
screen_height = desktop.height() # Get screen height
# Use absolute positioning to place the GUI in the center of the screen
self.move((screen_width - self.width()) / 2, (screen_height - self.height()) / 2)
```

Summary

By taking you through some actual examples of programs with working menus, my hope is that you can see how many classes are working together just to make a single application. The examples in this chapter are by no means all that can be done with menus. There are still plenty of other ways to organize the actions and widgets in your own projects including context menus (often referred to as pop-up menus), tabbed wid-

gets with the `QToolBox` class, stacked widgets using the `QStackedWidget` class, and more.

Chapter 5 focused on the `QMainWindow` class for integrating menus easily into GUIs. A menubar consists of several menus, each of which is broken down into several commands. Each of these commands could themselves also be checkable or even submenus. Toolbars are often composed of icons that allow the user to more easily locate commands. The `QDockWidget` class creates movable and floating widgets that can be used to hold a number of different tools, widgets, or commands. Finally, the status bar created from the `QStatusBar` class establishes a space to give further textual information about each of the menu items.

The class that acts like the glue to keep track of all the different functions and whether they have been triggered or not is the `QAction` class. The `QAction` class manages these actions to ensure that no matter where the action is triggered, whether from a menu, the toolbar, or from shortcut keys, the application can perform the next appropriate action.

In Chapter 6 we will see how to modify the appearance of widgets with style sheets and learn about how to create custom signals in PyQt.

Footnotes

¹ <https://doc.qt.io/qt-5/qprinter.html#OutputFormat-enum>
